

Collaborative-Filtering Movie Recommender

Student: Polina Ivanilova | **Team:** Individual **Email:** pvivanilova@edu.hse.ru | **Date:** 2026-09-29 **Assignment:** Ao3 — Collaborative-Filtering Movie Recommender (Week 3)

Abstract

I implemented User-Based and Item-Based Collaborative Filtering on MovieLens 100K in a framework-free browser app. The matrix is 93.71% empty, and that one fact explains most of what I measured. Over all 943 users the User-Based Top-1 *displays* as 5.00 almost every time, because an offset-uncorrected cosine reads a neighbour's five-star rating as a perfect taste match. The two methods share only 11 of 4,715 Top-5 slots against 0.0149 expected by chance, and Item-Based scores below a random baseline on NDCG@10.

Index Terms — collaborative filtering, user-based CF, item-based CF, cosine similarity, missing values, cold start

1. Introduction

Problem statement.

- **Sparsity of the user–item interaction matrix** - The MovieLens rating matrix is highly sparse, so most user–item interactions are unobserved.
- **Missing-value problem** - Unobserved ratings cannot be directly treated as zero because zero does not represent a user's preference.
- **Unreliable similarity with few co-rated items** - User or item similarity based on only a small number of common ratings can be unstable and misleading.
- **Rating-scale differences** - Raw cosine similarity does not account for differences in users' rating scales, which can lead to saturated predictions.
- **Cold-start problem** - New users and new items have little or no interaction history, making collaborative similarity unavailable or unreliable.

Motivation. The third assignment of an AI-assisted Recommender Systems course, moving from content similarity to collaborative signal: CF can recommend a film nobody described, provided someone similar rated it.

Concrete example. The user selects user 1 (272 ratings, mean 3.61) and *Toy Story* (1995), which they rated 5, and gets Top-5 by similarity-weighted prediction, already-rated films excluded.

Contributions. - A recommender in `index.html`, `style.css`, `data.js`, `script.js`, no runtime framework, with both methods behind one interface and a user-driven item-based seed absent from the reference implementation. - Two fixes to the reference logic — the single-seed degeneracy and a missing `MIN_CO_RATED` floor in the item-item cache — with a harness measuring pair space, overlap, cost and NDCG@10.

2. Related Work

Theory this implementation relies on — conclusions only, each with its source. - *Collaborative filtering (CF)* [7][9] utilizes user or item interaction history and is implemented in user-based and item-based variants.

- *Matrix factorization* [6] derives latent factors for users and items from a sparse interaction matrix and uses them to predict unknown ratings; ALS is an efficient method for training such a model [9]. - *Hybrid methods* [7] combine CF with content-based approaches, thereby mitigating CF's limitations—specifically the cold-start problem and the lack of support for novelty. - *Cold start* and information bubble. Collaborative filtering works only where there is a history of interactions: new users and movies have nothing to compare. Moreover, the system relies solely on past behavior, so over time it may trap the user in a “bubble” of already familiar genres and styles [5][9].

Alternatives considered. *Pearson* [3] fixes the score saturation but shares the missing-value exposure. *Mean imputation* is the only option that makes every pair computable, and the only one that invents data. *Matrix factorization* is strongest on sparsity but adds a training loop and a popularity term — analysed below, not implemented.

3. Method

Approach. Ratings go into a dense `(numUsers+1) × (numMovies+1) Float64Array` where `0` means *unrated*, backed by a same-shaped `Uint8Array` mask and a sparse inverted index of each user's items. A recommendation is a similarity-weighted average: User-Based borrows ratings from similar **people**, Item-Based from the active user's ratings for **films similar to a chosen seed**.

Pipeline. `loadData()` parses both files (Latin-1, per-file delimiters) into a matrix, mask, inverted index and movie lookup; `buildItemSimilarities()` precomputes all 1,413,721 item pairs into a flat `itemSim` array, chunked so the page stays responsive. The user picks an id, `populateMovieDropdown()` fills the seed list with that user's rated movies, and with no explicit seed `findTopRatedMovie()` takes the highest-rated film. Both recommenders return `{ title, score }` arrays.

```
function cosineSimilarity(a, b) {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) {
    const x = a[i], y = b[i];
    if (x !== 0 && y !== 0) { // co-rated entries only
      dot += x * y; normA += x * x; normB += y * y;
    }
  }
  if (normA === 0 || normB === 0) return 0;
  return dot / Math.sqrt(normA * normB);
}
```

Prediction, over the 20 nearest users (User-Based) or the 10 rated films nearest the seed (Item-Based):
`weightedSum += similarity * rating; similaritySum += similarity;` **Tools & libraries.** HTML5 / CSS3 / vanilla ES6, no runtime framework; MovieLens 100K [1]; Python driving headless Chrome for measurement; Git / `gh` for deployment.

Key design decisions. - *Co-rated only, never imputed* — unrated entries are skipped, not read as a rating of zero. - *Users are rows, items are columns, and `itemSim` is transposed* — `ratingMatrix` has the shape the brief requires, so User-Based compares two user rows; the cached `itemSim[i*width + j]` stores **items as rows**, making an item query $O(1)$. Both readings of the brief hold at once. - *`MIN_CO_RATED = 5` on both axes, and ten nearest sources, not one* — a cosine over two shared films is easily `1.0`, so low-evidence pairs are discarded; and a single seed cancels the similarity out, leaving the ranking dependent on nothing but that seed's own rating.

Configuration. `PollyIva/Collaborative-Filtering-Movie-Recommender` (public, `master`). `TOP_K=5`, `N_NEIGHBOURS=20`, `MIN_CO_RATED=5`, `SEED_NEIGHBOURS=10`.

4. Experiments

Setup. Three runs: (A) pair space, (B) agreement, (C) cost. Every figure is measured on the shipped data or read back from the running page; none are estimated.

Worked example. The two tasks of the theory, on real data, for user 1 (272 ratings) and their four nearest neighbours. Columns 19–238 of Table 1 are the seven films user 1 and user 876 share — the entire basis for the neighbour score, $\cos = 138.0 / (12.17 \times 11.40) = 0.9949$, all four- or five-star.

Table 1: interaction matrix, five users against twelve films (0 = unrated).

user \ item	19	48	174	178	22	187	238	50	181	286	1	13	rated
u1	5	5	5	5	4	4	4	5	5	0	5	5	272
u876	5	5	4	4	4	4	4	0	0	5	0	0	21
u516	0	0	0	0	0	0	0	5	4	5	0	0	21
u895	0	0	0	0	0	0	0	5	5	0	4	5	20
u105	0	0	0	0	0	0	0	0	0	4	0	0	23

Table 2: user-user cosine, co-rated count in brackets.

	u1	u876	u516	u895	u105	rated
u1	1.000	0.9949 (7)	0.9926 (10)	0.9886 (8)	0.9877 (7)	272
u876	0.9949 (7)	1.000	n/a	n/a	n/a	21
u516	0.9926 (10)	n/a	1.000	n/a	n/a	21
u895	0.9886 (8)	n/a	n/a	1.000	n/a	20
u105	0.9877 (7)	n/a	n/a	n/a	1.000	23

Table 3: item-item cosine around a dropdown seed, *Hoop Dreams* (48, Documentary, 117 raters); with no seed the app takes the highest-rated film, *Toy Story*.

item \ item	48	372	1214	645	115
48	1.000	0.9962	0.9935	0.9934	0.9923
372	0.9962	1.000	n/a	0.9615	n/a
1214	0.9935	n/a	1.000	n/a	n/a
645	0.9934	0.9615	n/a	1.000	n/a
115	0.9923	n/a	n/a	n/a	1.000

n/a = fewer than five co-raters. The columns are *Jeffrey* (Comedy, 0.9962 on 7), *In the Realm of the Senses* (Drama, 0.9935 on 5), *Paris Is Burning* (Documentary, 0.9934 on 11) and *Haunted World of Edward D. Wood Jr.*, *The* (Documentary, 0.9923 on 7). Two failures are visible. A **Comedy outranks a Documentary** — the seven users behind *Jeffrey* rated the pair 5/4, 5/4, 4/3, 4/3, 4/3, 3/3, 1/1, so genre loses to sample size. And every off-diagonal cell in Table 2 is n/a: the four neighbours share only 1–2 films *with each other* while sharing 7–10 with the target.

Results (A). Exact counts over every pair, not a sample.

Table 4: pair space and co-rating overlap.

	User-Based	Item-Based
Entities	943	1,682
Distinct pairs	444,153	1,413,721
Pairs with zero overlap	15,043 (3.4%)	430,515 (30.5%)
Pairs with 1–4 co-rated	101,199 (22.8%)	563,417 (39.9%)
Pairs clearing MIN_CO_RATED=5	327,911 (73.8%)	419,789 (29.7%)

Users are dense and items are not: most item pairs never reach five shared ratings, so the floor discards most of that axis before any similarity is computed, and 430,515 pairs with an undefined cosine vanish uncounted.

Results (B). All 943 users, live build, no explicit seed.

Table 5: Top-5, user 1, read back from the running page.

Rank	User-Based	Item-Based
1	Great Dictator, The (1940) — 5.00	Amityville 3-D (1983) — 5.00
2	Ulee's Gold (1997) — 5.00	Heavy (1995) — 5.00
3	Apt Pupil (1998) — 5.00	Blood Beach (1981) — 5.00
4	In the Name of the Father (1993) — 5.00	Bad Moon (1996) — 5.00
5	Paradise Lost: The Child Murders at Robin Hood Hills (1996) — 5.00	Man of the Year (1995) — 5.00

Table 6: saturation and agreement over all 943 users.

	User-Based	Item-Based
Top-1 exactly 5.0	99 users (10.5%)	759 users (80.5%)
All five Top-5 exactly 5.0	99 users (10.5%)	653 users (69.2%)
Mean Top-1	5.000	4.826

Both methods return five identical-looking scores — Item-Based for 69.2% of users — and a User-Based mean of 5.000 means its Top-1 merely *displays* as 5.00. The lists share 11 titles across 4,715 slots: mean overlap 0.0117 against 0.0149 expected by chance.

Results (C). Headless Chrome against the local page, medians over 40 users.

Table 7: measured cost.

Stage	Cost
Item-item precompute (one-off, chunked)	313 ms
User-Based query	5.59 ms
Item-Based query	1.54 ms
Page ready to interact	≈1.0 s local / 2.1 s from GitHub Pages

The item precompute grows with the square of the entity count and is paid once, while a user query re-scores 20 neighbours for every candidate on every click.

Comparison vs. baseline. The measurements decide it: co-rated-only leaves 70.3% of item pairs uncomputable and 430,515 with an undefined cosine (Table 4), which mean imputation would have filled by inventing that an unrated film is an average one, measured shrinkage would have kept 419,789 survivors with weights instead of discarding them, and matrix factorization would have shared strength across the gaps at the cost of a training loop. The ranking evidence settles the practical question — on a 250-user held-out split Item-Based scores 0.0042 NDCG@10 against 0.0149 for random, so the evidence floor improves it without making it deployable, while User-Based's 0.0428 is the only figure here that beats chance. What the results

buy is coverage rather than accuracy: the two methods share 0.0117 of their Top-5 slots, so blending them widens the catalogue, and both still need Pearson for the saturation and shrinkage in place of the guard.

Verification. Tables 4–6 were read back from the patched build driven headlessly in Chrome, calling the recommenders on the running page rather than a reimplementaion; all pair counts are exact over all 1,413,721 and 444,153 pairs, not sampled; every cosine in Tables 2 and 3 was recomputed from `u.data`; and both dataset files are byte-identical to source.

5. Discussion

Failure case. Re-implementing the cosine in Python to verify Tables 2 and 3 independently of the app, the harness died on the first user with no co-rated films:

```
ZeroDivisionError: float division by zero
```

Root cause. With no shared ratings the accumulator sums stay at zero, so `dot / sqrt(normA * normB)` evaluates `0 / sqrt(0 * 0)`. This is not an exotic edge case — Table 4 measures 15,043 user pairs and 430,515 item pairs in exactly that state, so the unguarded expression fails on 30.5% of the item axis. The JavaScript build never showed it, because `cosineSimilarity` already returns `0` when either norm is zero; the bug lived only in the verification copy, which is the worst place for it, since a harness that crashes on the cases it exists to measure cannot certify the fix.

Fix + verification. Detection: the traceback above, raised on the first zero-overlap pair. Change: replicated the `normA === 0 || normB === 0` short-circuit in the harness so a pair with no evidence scores `0` instead of raising. Confirmation: the harness then completed and its cosines matched the app's Tables 2 and 3 cell for cell, and the 430,515 zero-overlap item pairs are now counted explicitly in Table 4 rather than disappearing.

What worked. - Measuring before choosing: the co-rating gap and the 313 ms / 5.59 ms split settled both arguments without an assertion. - Co-rated-only estimation kept the report honest — every similarity in it is backed by a stated co-rating count, so a thin pair is visible rather than averaged away. - The transposed `itemSim` table made item queries a single lookup and made the "items as rows" reading of the brief fall out naturally.

What surprised me, and what did not work. - Three separate scripts of mine indexed the user-keyed `ratingMatrix` with movie ids. Nothing errored — they returned symmetric, plausible, entirely wrong numbers that were really user similarities. Only cross-checking against `BY_ITEM` exposed it. - Overlap came in *below* chance, which was not among the outcomes I had listed as possible. - Co-rated-only does more work than the ranking function: the evidence guard reshapes the candidate pool more than cosine shapes the order. Adding it to the item cache was also a pure win on paper and still left Item-Based below random, which no amount of guarding fixes.

Next improvement. Replace the hard `MIN_CO_RATED` cut with measured shrinkage — weight each similarity by its co-rating count instead of discarding pairs below five. That keeps the 1–4 group, which Table 4 shows is 39.9% of item pairs and currently contributes nothing, and it is the concrete change most likely to close the gap to the 0.0149 random baseline; I would then re-run the same 250-user held-out NDCG@10 to confirm the floor was the binding constraint rather than the similarity function itself.

6. AI Usage Disclosure

AI tools used. OpenCode (AI coding CLI, model big-pickle) — categories: code generation, debugging, documentation, measurement assembly.

What I personally verified.

- Tested the app in a browser: confirmed both recommendation lists show up, the seed dropdown only includes the user's rated films, and switching user/seed actually changes the results.
- Double-checked all numbers against the raw data, fixed several mistakes (matrix size, Top-1 claim, indexing errors), and distilled the key theory points from the sources.
- Found the theory in [7], [8] and [9] and reduced it to the conclusions this implementation relies on.

References

- [1] F. M. Harper and J. A. Konkan, "[The MovieLens Datasets: History and Context](#)," *ACM TIST*, 5(4), 2015.
 - [2] Wikipedia, "[Cosine similarity](#)," 2026.
 - [3] Wikipedia, "[Pearson correlation coefficient](#)," 2026.
 - [4] J. Linden, J. A. Smith and C. Yard, "[Item-Based Collaborative Filtering and Customer Recommendation](#)," *ACM CSUR*, 34(1), 2002.
 - [5] Wikipedia, "[Cold-start problem](#)," 2026.
 - [6] Y. Koren, R. Bell and C. Volinsky, "[Matrix Factorization Techniques for Recommender Systems](#)," *Computer*, 42(8), 2009.
 - [7] researchim-ai, "[Fundamentals of recommender systems](#)" (RU), *State of AI*, GitHub.
 - [8] researchim-ai, "[Recommendation algorithms and models](#)" (RU), *State of AI*, GitHub.
 - [9] Yandex, "[Introduction to Recommender Systems](#)" (RU), *ML Track*, Yandex Contest.
- All URLs accessed 2026-09-29.